



Webvisualisierung von Prozesskomponenten in der Bildgebenden Qualitätskontrolle

Studienarbeit 2

des Studienganges Elektrotechnik
an der Dualen Hochschule Baden-Württemberg Mannheim

von
Andreas Braig

02.01.2025

Bearbeitungszeitraum:	07.01.2025 - 07.04.2025
Matrikelnummer, Kurs:	6481829, TEL22AT1
Ausbildungsfirma:	ABB AG
Abteilung:	PAPI-EAM
Betreuer der Dualen Hochschule:	Prof. Dr.-Ing. Bozena Lamek-Creutz

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: „Webvisualisierung von Prozesskomponenten in der Bildgebenden Qualitätskontrolle“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Unterschrift

Zusammenfassung

Die vorliegende Arbeit befasst sich mit einem Folgeprojekt der vorangestellten Studienarbeit Anwendung des Maschinellen Lernens in der bildgebenden Qualitätskontrolle und untersucht eine Verbesserung und Visualisierung der Ergebnisse der vorangestellten Studienarbeit.

Zunächst wird nach der zugrunde liegenden Theorie der Verwendung von Application Programming Interface (API)s das Konzept zur Realisierung der Aufgabenstellung vorgestellt.

Die Umsetzung erfolgt zunächst in einer vom Labor unabhängigen Programmierumgebung und implementiert alle geforderten Aufgabenteile erfolgreich. Nach ersten Tests wird das Programm auf die Laborsysteme übertragen und dort getestet.

Abschließend wird das Convolutional Neural Network (CNN) anhand eines neuen Datensatzes evaluiert und mit anderen Architekturen anhand der gängigen Metriken verglichen.

Die Ergebnisse dieser Arbeit liefern Erkenntnisse für die einfache Implementierung einer bidirektionalen Webserver-Applikation und zeigen eine einfache Visualisierungsmöglichkeit der Klassifikationsergebnisse von CNNs. Die Ergebnisse bieten Grundlagen für zukünftige Arbeiten im Bereich der Bildverarbeitung und der Qualitätssicherung im Industrieumfeld.

Abstract

This thesis deals with a follow-up project to the preceding study *Anwendung des Maschinellen Lernens in der bildgebenden Qualitätskontrolle* and examines an improvement and visualization of the results of the preceding study.

First, after discussing the underlying theory of using APIs, the concept for implementing the task is presented.

The implementation is initially carried out in a programming environment independent of the laboratory and successfully implements all required tasks. After initial tests, the program is transferred to the laboratory systems and tested there.

Finally, the CNN is evaluated using a new dataset and compared with other architectures based on common metrics.

The results of this work provide insights into the simple implementation of a bidirectional web server application and demonstrate an easy way to visualize the classification results of CNNs. The findings lay the groundwork for future work in the field of image processing and quality assurance in an industrial environment.

Inhaltsverzeichnis

Inhaltsverzeichnis	IV
Abbildungsverzeichnis	VI
Listingverzeichnis	VII
1. Problemstellung und Ziel dieser Arbeit	1
2. Theoretische Grundlagen	2
2.1. Machine Learning und Klassifikationsprobleme	2
2.2. Die Funktionsweise einer API	4
2.2.1. Kommunikation einer API	5
2.2.2. FLASK	6
3. Softwarekonzept	8
3.1. Programmstruktur	8
3.2. Konfiguration der Programmparameter	9
3.3. Die Weboberfläche mittels Python Web API	10
4. Implementierung	12
4.1. Umzug auf lokale Programmierung	12
4.2. Einpflegen der JSON-Parameter	14
4.3. Entwicklung der API	15
4.4. Installation im Labor	16
4.4.1. Aufgetretene Probleme	17
4.4.2. Neuer Datensatz	18
5. Softwaretests	20
5.1. Metriken zur Evaluierung	20
5.1.1. Accuracy und Loss	21
5.1.2. Confusion Matrix und F1 Score	22
5.2. Reevaluierung des Modells	23

6. Fazit und Ausblick	26
Literaturverzeichnis	X
A. Anhang	XII
A. Anleitung zur Verwendung	XII
A.1. Manuelle Archivierung	XIII
A.2. Automatische Archivierung	XIII
B. Performance der Modelle	XVI
B.1. Pytorch Custom CNN	XVI
B.2. MobileNet V3 (TensorFlow)	XVI
B.3. MobileNet V1 (TensorFlow)	XVI

Abbildungsverzeichnis

2.1. Aufbau eines Convolutional Neural Networks [2].	3
2.2. Schematischer Aufbau der API Kommunikation	4
2.3. Erweiterung der API Darstellung auf Basis von FLASK	6
3.1. Schematische Darstellung der MVC Struktur [13]	8
3.2. Konzept der Parametergruppierung- und Verteilung auf das Programm .	10
3.3. Aufschlüsselung der API Funktionen	11
4.1. Projektstruktur der Software im Dateiverzeichnis	13
4.2. Darstellung der Weboberfläche im Zustand ohne Klassifikation eines aktuellen Printed Circuit Board (PCB)s.	17
4.3. Gegenüberstellung der alten und neuen PCBs	18
5.1. Vergleich der Modelle anhand des gemittelten F1 Scores über beide Klassen	23
5.2. Vergleich der Modelle anhand der Confusion Matrizen	24

Listings

4.1. Beispiel einer JavaScript Object Notation (JSON)-Datei mit Parametern des mobilnet Modells	14
4.2. Einlesen der JSON-Datei	14
4.3. Start der Weboberfläche und der Threads in der api.py	15
4.4. JavaScript Code der Weboberfläche, welcher die Bilder des Websockets empfängt	16

Abkürzungsverzeichnis

CNN	Convolutional Neural Network
API	Application Programming Interface
DHBW	Duale Hochschule Baden-Württemberg
PCB	Printed Circuit Board
CP-Lab	Cyber-Physical Lab
JSON	JavaScript Object Notation
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
URL	Uniform Resource Locator
HTML	Hypertext Markup Language
WSGI	Web Server Gateway Interface
MVC	Model-View-Controller
PC	Personal Computer
USB	Universal Serial Bus
SPS	Speicherprogrammierbare Steuerung

1. Problemstellung und Ziel dieser Arbeit

Die zunehmende Automatisierung industrieller Prozesse erfordert zuverlässige Qualitätskontrollsysteme, insbesondere in der Fertigung von elektronischen Baugruppen wie PCBss. Im letzten Semester wurde in einer Machbarkeitsstudie untersucht, ob ein KI-basiertes System umsetzbar ist, welches mittels TensorFlow CNNs Defekte auf PCBss erkennt. Dieses System basiert auf einer Online Implementierung des Klassifikationsprogrammes.

Aktuell bestehen drei zentrale Herausforderungen: Erstens bietet die Online Implementierung keine zusammengefasste Kontrolle über Parameter oder Daten, was die Flexibilität limitiert. Zweitens fehlt eine intuitive Schnittstelle zur Visualisierung von Klassifizierungsergebnissen, was die Benutzerinteraktion erschwert. Drittens ist mit der Online Implementierung keine Klassifikation in Echtzeit möglich. Weiterhin soll die Leistung der bisher verwendeten CNN-Architektur evaluiert werden, und weitere Architekturen oder Optimierungstechniken sollen getestet werden, um die Erkennungsgenauigkeit zu verbessern. Hierfür steht eine Reihe neuer PCBs zur Verfügung, die in einer zu dieser Arbeit parallelen Studienarbeit entwickelt werden.

Ziel dieser Arbeit ist es, die bestehende Lösung in eine lokale Anwendung zu überführen, die folgende Kernkomponenten integriert: Eine zentrale Parametrisierung, eine Webanwendung mit einer Python API, die eine Darstellung in Echtzeit von PCBs Klassifizierungen bietet.

Durch diese Maßnahmen wird die Darstellung der industriellen Anwendbarkeit im FESTO CP Lab der Duale Hochschule Baden-Württemberg (DHBW) gestärkt. Die neue Webvisualisierung soll greifbar machen, was bildverarbeitende Qualitätskontrolle bedeutet, indem sie in Echtzeit Analyse und Ergebnisse der PCBs anschaulich darstellt. Benutzer können direkt sehen, dass Defekte erkannt und klassifiziert werden, was die Transparenz und Nachvollziehbarkeit des gesamten Prozesses erhöht.

2. Theoretische Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen erläutert, die für das Verständnis der späteren Kapitel notwendig sind. Dazu gehören die Funktionsweisen von Machine Learning und Computer Vision, sowie die Grundlagen einer API. Wie bereits im Ersten Kapitel (siehe Kapitel 1) beschrieben, baut diese Studienarbeit auf der Arbeit des letzten Semesters auf. Die theoretischen Grundlagen für Machine Learning und Computer Vision werden hier nur kurz erläutert, da sie bereits im letzten Semester ausführlich behandelt wurden.

2.1. Machine Learning und Klassifikationsprobleme

Bildverarbeitung beschäftigt sich mit der Verarbeitung und Analyse digitaler Bilder durch Algorithmen und bildet die Grundlage für komplexere Verfahren [1]. Ein digitales Bild wird als mehrdimensionale Matrix gespeichert. Farbige Bilder werden als dreidimensionale Tensoren dargestellt, deren Dimensionen repräsentieren Höhe, Breite und Farbkanaäle. Diese Repräsentation ermöglicht die Anwendung verschiedener Transformationen wie Filterung, Segmentierung oder geometrische Transformationen. Diese dienen entweder zur Bildverbesserung oder als Vorverarbeitungsschritt für nachfolgende Analysen durch fortschrittlichere Techniken wie Machine Learning und Computer Vision durch Neuronale Netzwerke [2].

Convolutional Neural Networks CNN sind spezialisierte Deep Learning Modelle, die für die Verarbeitung von Bilddaten optimiert sind. Sie bilden die Grundlage für zahlreiche moderne Computer Vision Anwendungen, wie Gesichtserkennung und autonome Fahrzeuge. Die Architektur eines CNN nutzt die räumliche Struktur von Bildern effizient, um visuelle Muster zu extrahieren und zu klassifizieren. Ein CNN besteht hauptsächlich aus Faltungsschichten und linearen Ebenen (siehe Abbildung 2.1). Die Convolutional Layers verwenden kleine Filtermatrizen (Kernel), die über das Bild gleiten und visuelle Merkmale wie Kanten und Texturen erkennen. Diese Filter werden während des Trainings optimiert, um die relevantesten Merkmale zu extrahieren [2] [3].

Convolutional Neural Network

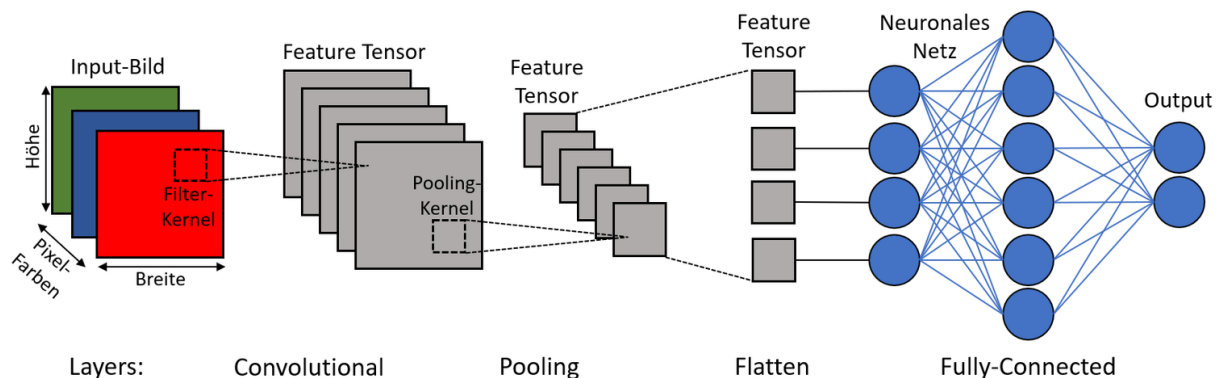


Abbildung 2.1.: Aufbau eines Convolutional Neural Networks [2].

Die Fully-Connected Layers am Ende des Netzwerks (siehe Abbildung 2.1) wandeln die extrahierten Merkmale in eine endgültige Klassifikation oder Vorhersage um. Diese Schichten sind ähnlich wie neuronale Netzwerke aufgebaut, welche nicht der Bildverarbeitung dienen. Hier ist jedes Neuron mit allen Neuronen der vorherigen Schicht verbunden.

Die wichtigsten Lernansätze im maschinellen Lernen sind überwachtes, unbeaufsichtigtes und verstärkendes Lernen. In dieser Arbeit wird überwachtes Lernen genutzt, bei dem ein Algorithmus mit gelabelten Daten trainiert wird, um aus diesen Beispielen zu lernen und Vorhersagen zu treffen.

Beim überwachten Lernen gibt es zwei Hauptmodelle: Klassifikation und Regression. Regression beschreibt kontinuierliche Zusammenhänge zwischen Eingangs- und Ausgangsdaten [4]. Klassifikation teilt Daten in diskrete Gruppen ein. Es sollen keine kontinuierlichen Werte nachgebildet werden. Am Ende des Netzwerks wird mithilfe einer $\text{argmax}()$ Funktion eine Klasse fest zugeordnet [5, S. 450]. Die binäre Klassifikation, welche in dieser Arbeit angewandt wird ist eine spezielle Form der Klassifikation, bei der nur zwei Klassen unterschieden werden.

Es gibt verschiedene CNN-Architekturen wie ResNet und MobileNet, die für spezifische Aufgaben besonders gut geeignet sind. Oft werden vortrainierte Modelle verwendet und für spezifische Anwendungsfälle abgestimmt. Diese Technik bekannt als Transfer Learning, basiert auf dem überwachten Lernen basiert und wird hier eingesetzt [2].

2.2. Die Funktionsweise einer API

Anwendungsprogrammierschnittstellen APIs sind Software-Vermittler (Abbildung 2.2) und stellen das Herzstück moderner Softwareentwicklung dar. Sie ermöglichen es, Programme miteinander zu kommunizieren und Daten auszutauschen. Dieser Datenaustausch ist wesentlich für vielfältige Anwendungen, wie etwa das Abrufen von Wetterdaten oder das Interagieren mit sozialen Netzwerken innerhalb einer fremden Anwendung. In Python machen Bibliotheken wie "requests" oder "http.client" den Einstieg in die API Entwicklung besonders zugänglich [6]. Die in dieser Studienarbeit verwendete FLASK Bibliothek ermöglicht es, eine komplexere API zu erstellen, die Daten aus dem Python Programm an die Website überträgt. APIs sind Vermittler sind allgegenwärtig im täglichen Leben, ob bewusst wahrgenommen oder nicht [7].

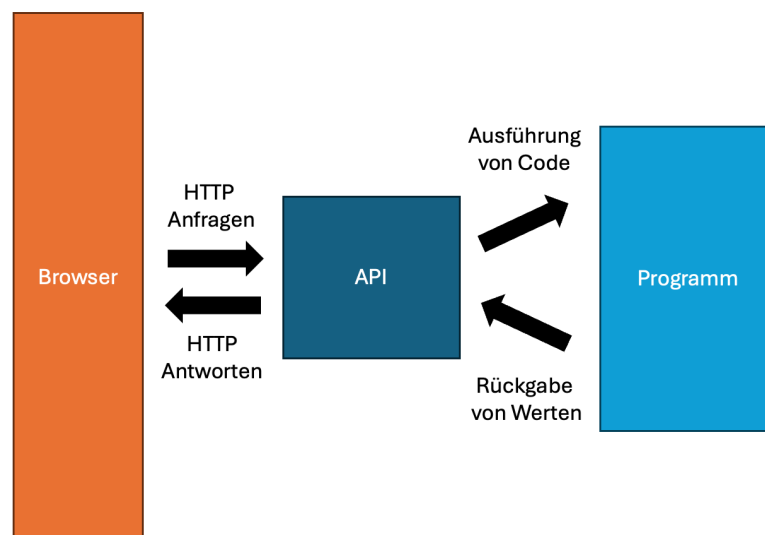


Abbildung 2.2.: Schematischer Aufbau der API Kommunikation

Im Kontext von Webanwendungen bieten APIs den Zugriff auf Daten und Funktionalitäten von Drittanbietern. Dies ermöglicht es, Anwendungen um Funktionen wie Wetterinformationen, Sportergebnisse, Filmlisten, Tweets, Suchmaschinenergebnisse und Bildverarbeitung zu erweitern, ohne diese selbst entwickeln zu müssen. Ein Beispiel hierfür ist die Google Maps API, welche das Integrieren von Karten und Standortdaten in Anwendungen ermöglicht. Die Eigenentwicklung solcher Funktionen würde erhebliche Ressourcen beanspruchen, während die Nutzung von APIs eine schnelle und effiziente Integration ermöglicht[8].

2.2.1. Kommunikation einer API

Die Grundbausteine der Kommunikation einer API sind Anfragen (Requests) und Antworten (Responses). Diese basieren oft auf dem Hypertext Transfer Protocol (HTTP)-Protokoll, welches die Grundlage des modernen Internets liefert [6]. Jedes Mal, wenn ein Benutzer eine Website aufruft, initiiert der Browser mehrere Anfragen an den Server der entsprechenden Webseite, um die notwendigen Ressourcen für die Darstellung der Seite abzurufen. Der Server antwortet dann mit allen Daten, die für die Darstellung der Seite erforderlich sind, woraufhin der Browser die Seite darstellt [8].

Der generische Prozess der Python API-Kommunikation ähnelt der Kommunikation des HTTP-Protokolls. Er lässt sich wie folgt beschreiben: Ein Client, in dieser Studienarbeit der Browser des Laborrechners, sendet Daten an eine Uniform Resource Locator (URL). Der Server unter dieser URL liest die Daten und entscheidet was damit zu tun ist. Intern wird das passende Python Skript ausgeführt und gibt eine Antwort in form der HTML Datei Abbildung 4.2 an den Client zurück. Schließlich verarbeitet der Client die empfangenen Daten entsprechend seiner Programmlogik und stellt die Klassifikationsumgebung dar [8].

Ein wesentlicher Teil der Anfrage ist die Hypertext Transfer Protocol HTTP-Methode. Einige der gebräuchlichsten Methoden sind [9]:

- **GET** Dient dem Abrufen von Daten, ohne Änderungen auf dem Server vorzunehmen
- **POST** Wird verwendet, um neue Daten an den Server zu senden
- **PUT** Aktualisiert vorhandene Daten auf dem Server
- **DELETE** Entfernt Daten vom Server

Diese Methoden bilden die Grundlage des Representational State Transfer (REST)ful API-Designs, das in modernen Webanwendungen weit verbreitet ist [9].

2.2.2. FLASK

Für diese Studienarbeit ist keine umfangreiche WEB-API notwendig. Es soll lediglich eine einzige Hypertext Markup Language (HTML) Datei von der Python Anwendung an den Browser übertragen werden. Es ist für diese Anforderung kein Größeres Framework wie beispielsweise Django notwendig. FLASK ist ein Mikroframework für Python, das sich auf einfache und schnelle Entwicklung konzentriert [10]. Es ist besonders gut geeignet für kleine bis mittelgroße Projekte, bei denen die Verwendung eines größeren Frameworks übertrieben wäre. FLASK bietet eine Vielzahl von Erweiterungen, die die Entwicklung von Webanwendungen erleichtern. Es ist einfach zu erlernen und bietet eine Vielzahl von Funktionen, die für die Entwicklung von Webanwendungen erforderlich sind.

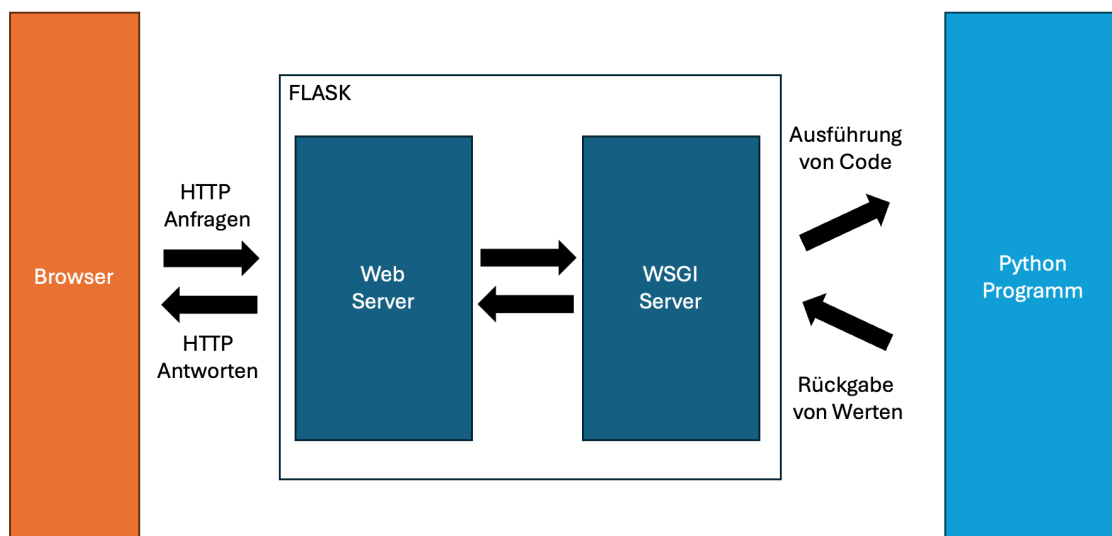


Abbildung 2.3.: Erweiterung der API Darstellung auf Basis von FLASK

FLASK nutzt die Web Server Gateway Interface (WSGI) Schnittstelle von Python [10] und Abbildung 2.3 und stellt in Verbindung mit der in dieser Studienarbeit verwendeten Waitress Bibliothek den Webserver zur Verfügung.

Eine besondere Anforderung der im folgenden vorgestellten Webanwendung liegt in der Übertragung der neu erstellten Fotos von der Kamera des FESTO Cyber-Physical Lab (CP-Lab) an die Webanwendung. Um eine einfache Benutzbarkeit zu gewährleisten, soll der Browser nicht jedes mal neu geladen werden müssen, wenn ein neues

Bild erstellt wird. Dies wird durch die Verwendung von Websockets gelöst.

Ein WebSocket ist eine Technologie, die es ermöglicht, eine bidirektionale Verbindung zwischen einem Client und einem Server herzustellen. FLASK SocketIO ist eine Erweiterung für FLASK, die die Verwendung von Websockets in FLASK Anwendungen ermöglicht. Diese Erweiterung wird verwendet, um die Übertragung der Fotos zu realisieren. Auf der Gegenseite im HTML Code wird die Bibliothek SocketIO.js verwendet. Diese basiert auf der Programmiersprache JavaScript und ermöglicht die Kommunikation zwischen dem Browser und dem Server.

Im Folgenden wird die neue Softwarearchitektur vorgestellt und auch die Implementierung der Webanwendung erläutert.

3. Softwarekonzept

Diese Studienarbeit liegt der in der letzten Studienarbeit beschriebene Code zugrunde. Zentraler Fokus des neuen Softwarekonzepts liegt auf der Präsentierbarkeit und Wiederverwendbarkeit dieser Machbarkeitsstudie als Anwendung für das CP-Lab.

Um die Aufgabenstellung aus Kapitel 1 zu erfüllen, ist es unumgänglich die Online-Implementierung durch eine lokale Anwendung zu ersetzen. Dies bringt eine größere Auswahl an Möglichkeiten sowie eine bessere Kontrolle über die Software mit sich. Für die bessere Kontrolle sollen alle einstellbaren Parameter zentral in einer Datei aufgeführt werden. Dies ermöglicht, eine einfache Konfiguration der Software und entspricht dem Stand der Technik [11] [12].

3.1. Programmstruktur

Der erster Teil der Softwarekonzeption ist der Entwurf der neuen Programmstruktur. Diese ist entscheidend für die Wartbarkeit und Erweiterbarkeit. Zwei zentrale Kriterien, bei denen die Erhaltung im gesamten Entwurfsprozess berücksichtigt werden muss.

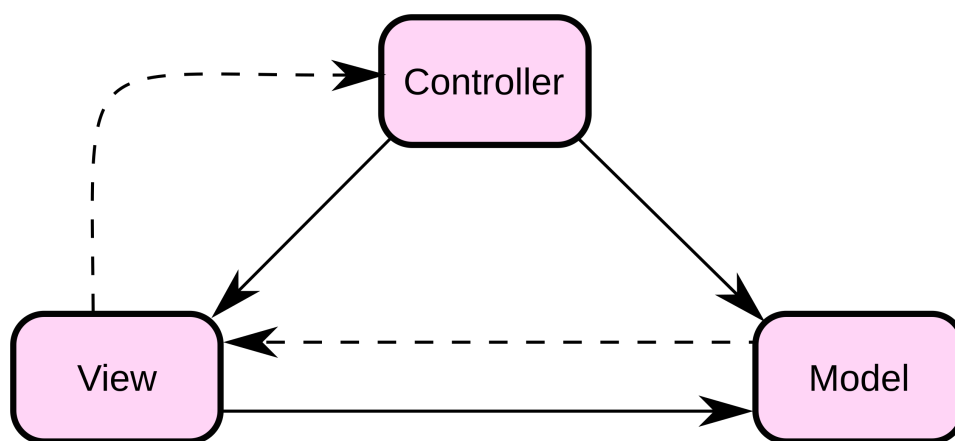


Abbildung 3.1.: Schematische Darstellung der MVC Struktur [13]

Für das neue Programm gibt es keine spezifischen Anforderungen an die Struktur.

Daher wird eine vereinfachte Model-View-Controller (MVC) Struktur gewählt (Abbildung 3.1). Ziel dieser Struktur ist es, die Software in drei Teile zu unterteilen. Diese drei Teile sollen eigenständige Aufgaben übernehmen und so verhindern, dass das Programm zu einem monolithischen Codeblock wird.

Der Model-Teil ist für die (Bild-) Datenverarbeitung zuständig und enthält die Datenstrukturen sowie die Logik, die die Daten verarbeitet. Der View-Teil ist für die Darstellung der Daten verantwortlich und umfasst die Benutzeroberfläche sowie die Logik, welche die Daten darstellt. Der Controller-Teil steuert die Daten und koordiniert die Kommunikation zwischen Model und View, indem er die entsprechende Logik enthält.

Die vereinfachte Version der MVC Struktur kombiniert die Funktionalitäten von View und Controller in der API (siehe Abbildung 3.1) und trennt die Datenverarbeitung in ein eigenes Modul ab. Dieses eigene Modul wird im Programmcode `Pycore` genannt und enthält die Funktionen, welche die Daten für das neuronale Netzwerk aufbereiten und das Klassifikationsergebnis zur Verfügung stellen.

3.2. Konfiguration der Programmparameter

Aus der Aufgabenstellung (siehe Kapitel 1) ist ein weiterer Teil des Softwarekonzepts, die Einstellbarkeit abzuleiten. Durch die Einstellbarkeit muss gewährleistet sein, dass die Software flexibel an die Anforderungen des Benutzers angepasst werden kann. Ohne dass der Benutzer ein tiefes Verständnis der Softwareimplementierung haben muss.

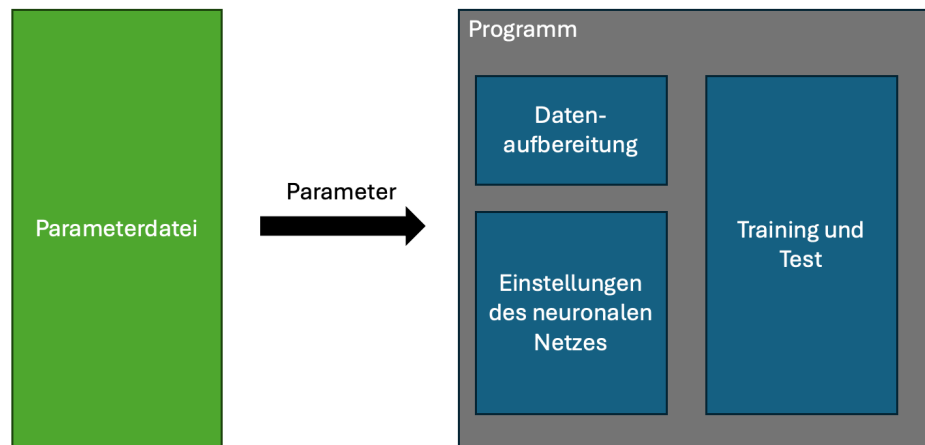


Abbildung 3.2.: Konzept der Parametergruppierung- und Verteilung auf das Programm

Ziel ist es, dass der Benutzer eine einzige Datei öffnet und dort alle Einstellungen vornehmen kann. Diese Datei soll im JSON-Format vorliegen, da es ein weit verbreitetes Format ist und von vielen Programmiersprachen unterstützt wird [11].

3.3. Die Weboberfläche mittels Python Web API

Als letzter Teil des Softwarekonzepts wird die Weboberfläche betrachtet. Das zentrale Element des neuen Programmentwurfs, etabliert das Interface zwischen Benutzer und Programm. Es ermöglicht neue Bilder des FESTO CP-Lab darzustellen, zu klassifizieren und die Ergebnisse auf seiner Weboberfläche zu visualisieren. Zusätzlich koordiniert sie die Überwachung des Ordners für neue Bilder und führt bei einem neuen Bild die Klassifikation dieser Bilder in defekt oder nicht defekt durch.

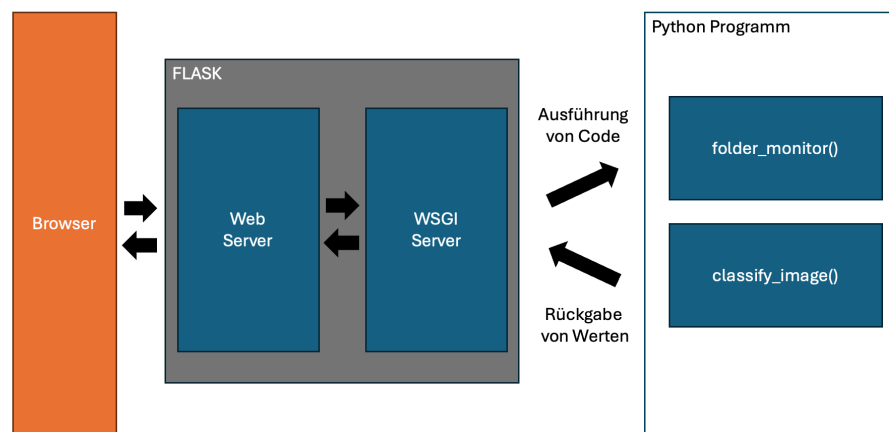


Abbildung 3.3.: Aufschlüsselung der API Funktionen

4. Implementierung

Nachdem im letzten Kapitel (siehe Kapitel 3) das Konzept zur Lösung der Aufgabenstellung beschrieben wurde, wird in diesem Kapitel die Implementierung der Software erläutert. Ziel ist es, alle beschriebenen Aufgabenteile in Python umzusetzen und eine Software zu entwickeln, die in der Lage ist, die Anforderungen der Aufgabenstellung zu erfüllen.

Die Implementierung erfolgt in mehreren Schritten:

1. **Umzug auf lokale Programmierung**
2. **Einpfelegen der JSON-Parameter**
3. **Entwicklung der API**
4. **Installation im Labor**

In den folgenden Kapiteln werden die einzelnen Schritte genauer beschrieben.

4.1. Umzug auf lokale Programmierung

Die Struktur dieser Implementierung folgt der Konzeption der Programmstruktur nach dem MVC-Prinzip Abbildung 3.1, welches in Abschnitt 3.1 beschrieben ist. Dieses Modell trennt die Datenverarbeitung, die Darstellung und die Steuerung der Software in drei verschiedene Module auf. Die Datenverarbeitung wird in einem eigenen Modul abgetrennt. Darstellung und Steuerung sind in der API zusammengefasst.

Das Pycore-Modul repräsentiert die Datenverarbeitung. Dieses Modul stellt die benötigten, selbst entwickelten, Bibliotheken zur Verfügung. Es ist in Abbildung 4.1 als eigener Ordner mit Python-Skripten und seinen Unterverzeichnissen dargestellt. Es beinhaltet unter Anderem die Funktionen, welche den Trainingsdatensatz aufbereiten

und dem neuronalen Netzwerk zum Training zur Verfügung stellen. Die wichtigsten Bestandteile sind die Konvertierung in Graustufen, die Aufteilung des Datensatzes in Trainings- und Testdaten sowie der Download des untrainierten neuronalen Netzwerks und dessen weitere Parametrisierung bezüglich der Klassifikationsaufgabe.

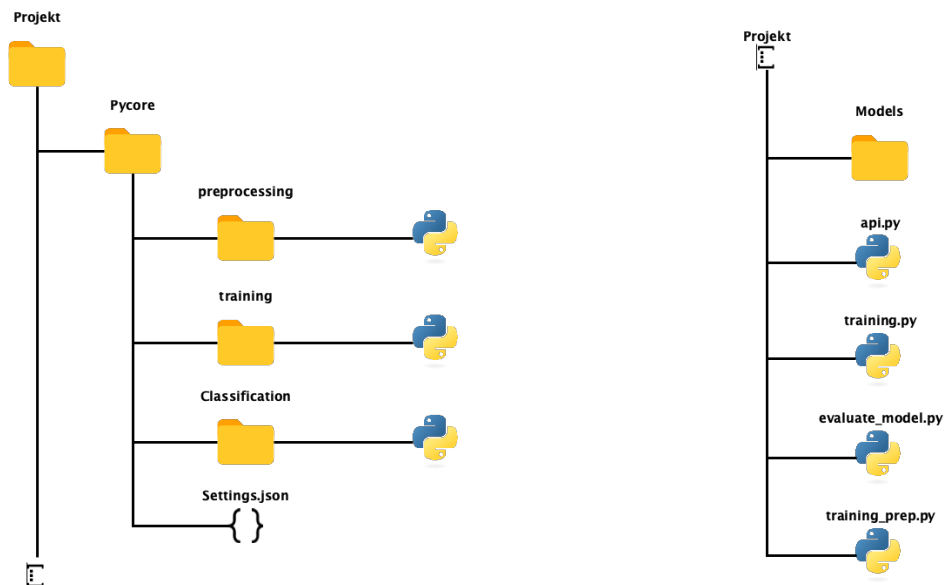


Abbildung 4.1.: Projektstruktur der Software im Dateiverzeichnis

Die in der Grafik Abbildung 4.1 dargestellte Dateistruktur repräsentiert ebenso die Struktur der Software. Die Funktionen werden in Bibliotheken im Pycore-Verzeichnis zusammengefasst. Sämtliche Daten werden in einem ausgegliederten Verzeichnis innerhalb des Projektverzeichnisses abgelegt.

Im Projekthauptverzeichnis gibt es, neben des Pycore-Moduls und der Verzeichnisse für Trainings- und Testdaten, vier Python-Skripte. Diese können ausgeführt werden, um einzelne Teile der Software auszuführen. `api.py` wird ausgeführt, um die Weboberfläche zu starten. `train.py` wird ausgeführt, um das Modell zu trainieren. `evaluate_model.py` wird ausgeführt, um das Modell zu evaluieren und die in Unterabschnitt 5.1.2 beschriebenen Werte zu erhalten. `training_prep.py` kann ausgeführt werden, um den Datensatz zu verarbeiten, ohne das Modell zu trainieren.

Diese Skripte laden den jeweils benötigten Programmteil und die benötigten Parameter aus dem Pycore-Verzeichnis, führen die entsprechenden Funktionen aus und legen die Ergebnisse im Dateisystem ab. Keines dieser Skripte ist über die Webanwendung

startbar, da sie nur für die interne Verwendung gedacht sind.

4.2. Einpflegen der JSON-Parameter

Die Umsetzung des Konzepts zur Konfiguration der Programmparameter (siehe Abschnitt 3.2) erfolgt in Python mittels einer JSON-Datei. Diese Datei enthält alle Parameter, die für die Software benötigt werden.

```
1  {
2      "filepaths": {
3          "good": "Bilder/Good_Pictures",
4          "bad": "Bilder/Bad_Pictures",
5          "good_gray": "Bilder/Good_Grayscale",
6          "bad_gray": "Bilder/Bad_Grayscale",
7          "train": "Bilder/train",
8          "test": "Bilder/test",
9          "validate": "Bilder/validate",
10         "new": "Bilder/new"
11     }
12 }
```

Listing 4.1: Beispiel einer JSON-Datei mit Parametern des mobilnet Modells

Die Struktur der JSON-Datei ist in Listing 4.1 dargestellt und wird in Python mittels des `json` Moduls eingelesen. Der hier dargestellte Ausschnitt beinhaltet alle wichtigen Dateipfade, um dem Python-Programm Zugriff zu den Trainings- und Testdaten sowie dem überwachten Ordner für neue Bilder zu gewähren. Er dient beispielhaft für das gesamte Programm. Ein Beispiel für das Einlesen der Datei ist in Listing 4.2 dargestellt.

```
1  import json
2
3  config_path = "pycore/setings.json"
4  cf = json.load(open(config_path, 'r'))
5
6  # Uebergabe der Parameter an die Funktionen
7  uic.folder_to_grayscale(cf["filepaths"]["good"], cf["filepaths"]["good_gray"])
8  uic.folder_to_grayscale(cf["filepaths"]["bad"], cf["filepaths"]["bad_gray"])
```

Listing 4.2: Einlesen der JSON-Datei

In diesem Codeausschnitt wird der Pfad zur Konfigurationsdatei festgelegt (Zeile 3) und die Datei wird eingelesen (Zeile 4). Die Parameter werden dann an die erste Funktion übergeben, welche die Bilder in Graustufen umwandelt und in einem separaten

Verzeichnis ablegt (Zielverzeichnis siehe Listing 4.2). Die Syntax für das Abrufen des Parameters ist `cf["filepaths"]["good"]`, wobei `cf` die Variable ist, in der die JSON-Datei eingelesen wurde. Die JSON-Datei wird als Array eingelesen, wobei der Zugriff auf die einzelnen Parameter über den Schlüssel in Textform erfolgt. In diesem Fall wird der Pfad zum Ordner mit den guten Bildern über den Schlüssel "good" angesprochen, während "filepaths" den Zugriff auf die passende Sektion des Arrays ermöglicht.

Angenommen, der Benutzer wünscht ein anderes Verzeichnis für die Bilder, so kann er dies in der JSON-Datei ändern und die Software erneut ausführen, ohne zu wissen, wo die Funktion, die den Datensatz generiert, abgelegt ist. Der gesamte Code dieses Projekts wurde nach diesem Schema aufgebaut, um eine einfache und einheitliche Konfiguration zu gewährleisten.

4.3. Entwicklung der API

In diesem Abschnitt wird die Entwicklung der API beschrieben, welche die Kommunikation zwischen der Weboberfläche und der Backend-Logik der Software ermöglicht. Es wird sichergestellt, dass die Entwicklung dem Konzept der Aufgabenstellung Abschnitt 3.3 entspricht.

Hierfür werden in zwei verschiedenen Threads zwei Funktionen ausgeführt. Die erste Funktion lädt das trainierte Modell in den Zwischenspeicher, um es für die Klassifizierung nutzbar zu machen (Zeile 3 Listing 4.3). Im zweiten Thread wird der Watchdog gestartet, welcher den Ordner für neue Bilder überwacht und bei neuen Bildern die Klassifizierung startet (Zeile 6).

```
1  if __name__ == '__main__':
2
3      model_thread = threading.Thread(target=load_model, daemon=True)
4      model_thread.start()
5
6      watchdog_thread = threading.Thread(target=start_watchdog, daemon=True)
7      watchdog_thread.start()
8
9      socketio.run(app, host="127.0.0.1", port=5000, debug=True, use_reloader=False)
```

Listing 4.3: Start der Weboberfläche und der Threads in der api.py

Ziel des Threadings ist es, den Ordner für neue Bilder permanent zu überwachen, ohne zyklisch abzufragen. Dies spart Rechenleistung und ermöglicht eine schnellere Reaktion auf neue Bilder [14].

Parallel zu den beiden Threads wird die Weboberfläche in Zeile 9 des Listing 4.3 gestartet. Bei Aufruf der Adresse, welche ebenfalls in Zeile 9 dargestellt ist, wird eine vorher definierte HTML-Seite an den Browser des Clients mittels GET Anfrage (siehe Unterabschnitt 2.2.1) gesendet. Diese Seite enthält ein JavaScript, welches die Verbindung zum Websocket des Python-Programms herstellt und einen bidirektionalen Datenaustausch ermöglicht.

```
1      <script>
2          var socket = io();
3          socket.on('update_image', function(data) {
4              console.log("Neues Bild-Event empfangen:", data); // Debugging
5              document.getElementById('latestImage').src = data.image_url + "?t=" + new Date
6                  ().getTime();
7          });
8          socket.on('classification_result', function(data) {
9              document.getElementById("classification-result").innerText = data.result;
10             });
11     </script>
```

Listing 4.4: JavaScript Code der Weboberfläche, welcher die Bilder des Websockets empfängt

Sobald ein neues Bild in den Ordner `Bilder/new` gelegt wird, wird ein Event ausgelöst, welches sowohl eine Klassifizierung herbeiführt als auch das Bild mittels Websocket an das JavaScript-Backend der Weboberfläche sendet.

4.4. Installation im Labor

Die Installation verläuft in drei Phasen: Übertragung, Installation und Tests. Zunächst wird die Software auf einem Universal Serial Bus (USB)-Stick in das Labor gebracht und auf den Laborrechner kopiert. Der Laborrechner ist Teil des Lieferumfangs des FESTO CP-Lab Systems und steht dieser Studienarbeit zur Verfügung.

Dieser Personal Computer (PC) wird ausschließlich für die Steuerung der FESTO-Anlage verwendet und verwaltet die Datenbank, die Konfiguration der Stationen und

die Programmierung der Speicherprogrammierbare Steuerung (SPS)en. Python ist auf diesem Rechner bereits installiert, jedoch fehlen die benötigten Pakete für die Software. Alle benötigten Pakete werden in einem Python Virtual Environment installiert, um die Installation dieser Studienarbeit zu isolieren und die Funktionalität des bestehenden Systems nicht zu beeinträchtigen [15].

Um eine Speicherung der von der Kamera aufgenommenen Bilder zu ermöglichen, muss innerhalb der Konfigurationssoftware der Kamera ein Archivierungspfad festgelegt werden. Dieser Pfad wird in der Software als Zielverzeichnis für die Klassifizierung genutzt.

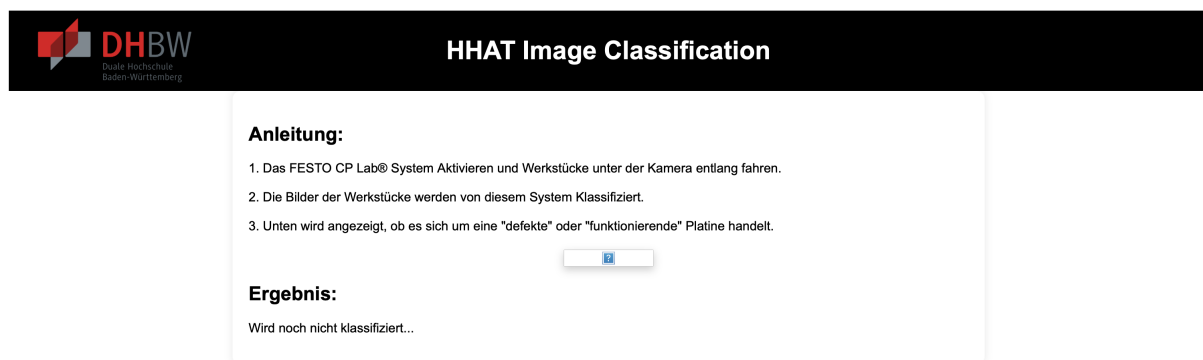


Abbildung 4.2.: Darstellung der Weboberfläche im Zustand ohne Klassifikation eines aktuellen PCBs.

Nach der Installation der benötigten Pakete wird die Software getestet. Die Interpretation des übertragenen Python-Codes ist fehlerfrei und der Testbetrieb kann aufgenommen werden. Die finale Gestaltung der Weboberfläche ist in Abbildung 4.2 dargestellt.

4.4.1. Aufgetretene Probleme

Während der Installation und des Testbetriebs traten zwei Probleme auf, deren Lösung im Folgenden erklärt wird.

Im Testbetrieb fällt auf, dass die Software Bilder nicht korrekt an die Webanwendung übermittelt werden. Der Grund für das Auftreten dieses Fehlers war ein überschnelles Reagieren des Watchdogs auf neue Bilder. Dieser erkennt eine Datei, bevor sie vollständig in den Ordner übermittelt wurde. Dieses Problem wird durch eine Verzögerung

der Bildübermittlung an den Webserver und den Klassifikator behoben. Die Übermittlung startet eine halbe Sekunde nach dem Auslösen des Events für ein neues Bild.

Der Vision-Sensor, welcher ebenfalls im Lieferumfang des FESTO CP-Lab Systems enthalten ist, kann mittels der Visor Vision Software verbunden werden [16]. Diese Software ermöglicht eine Anzeige und Konfiguration des Sensors über den Laborrechner. Eine permanente automatisierte Übertragung mittels direkter Verbindung zwischen Laborrechner und Vision-Sensor ist nicht möglich, da dies die Konfiguration des Sensors und damit die Funktionalität des CP-Lab Systems beeinträchtigen würde.

Um diesen Umstand zu umgehen, wird die Archivierung in der SensoView Software aktiviert. Der hierdurch entstehende Nachteil ist, dass die SensoView Software gestartet und angemeldet sein muss, um die Bilder zu speichern (siehe Anhang Abschnitt A). Zum jetzigen Stand ist eine permanente Lösung ohne Hilfe von FESTO nicht möglich.

4.4.2. Neuer Datensatz

Parallel zu dieser Studienarbeit wurde eine weitere Studienarbeit durchgeführt, welche sich mit dem Design neuer PCBs beschäftigt. Die detaillierten Änderungen und der Hergang des Designprozesses sind in der Studienarbeit von Herrn Lucas Weyland beschrieben und können dort nachgelesen werden. In dieser Studienarbeit wird diese Änderung als gegeben betrachtet und der neue Datensatz für die Klassifikation verwendet. Entsprechend dazu müssen neue Trainingsdaten generiert und das Modell neu trainiert werden.

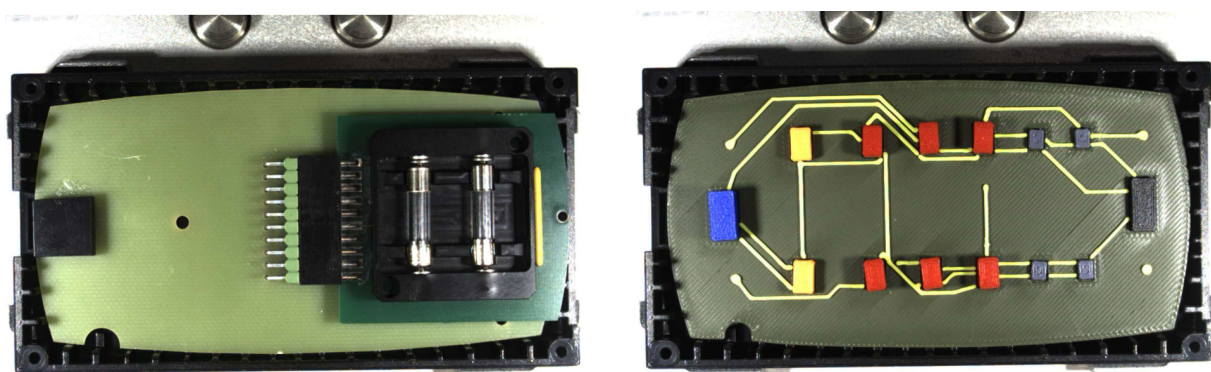


Abbildung 4.3.: Gegenüberstellung der alten und neuen PCBs

Wie auch auf der Abbildung 4.3 zu sehen ist, bieten die neuen PCBs (Rechts) weit

mehr Details und sind daher komplexer als die PCBs, die für die Machbarkeitsstudie verwendet wurden (Links). Dies führt zu mehr Möglichkeiten, die Grenzen des Modells zu testen und die Klassifikation zu verbessern. Die Reevaluierung des Modells mit den neuen Daten wird in Abschnitt 5.2 beschrieben.

5. Softwaretests

In diesem Kapitel wird auf die Evaluierungs- und Testvorgänge für die Sicherstellung der Güte der entwickelten Software eingegangen. Da es sich bei der entwickelten Software um ein Machine-Learning-Modell handelt, wird in diesem Kapitel auf die Evaluierung des Modells eingegangen. Die Funktionen des interpretierten Python Skripts entwickeln keine neuen Algorithmen, die auf ihre Fehleranfälligkeit oder Korrektheit getestet werden müssen.

5.1. Metriken zur Evaluierung

Um zu verstehen wie ein Machine Learning Modell evaluiert werden kann ist es zunächst wichtig die Arten von Kriterien nachzuvollziehen, die für die binäre Klassifikation notwendig sind. Im Folgenden wird allgemein von Instanzen gesprochen, es handelt sich dabei um die Bilder, der PCBs, die klassifiziert werden sollen.

- **True Positive (TP):** Die Anzahl der korrekt klassifizierten positiven Instanzen.
- **True Negative (TN):** Die Anzahl der korrekt klassifizierten negativen Instanzen.
- **False Positive (FP):** Die Anzahl der falsch klassifizierten positiven Instanzen.
- **False Negative (FN):** Die Anzahl der falsch klassifizierten negativen Instanzen.

Diese wesentlichen Typen teilen die Klassifizierten Daten nach dem Test in vier diskrete Kategorien ein. Diese Kategorien bilden die in Unterabschnitt 5.1.2 beschriebene Confusion Matrix. Mit Ihnen können aber auch die einfacheren Werte Loss und Accuracy berechnet werden. Im Folgenden wird die Mathematik hinter den Methoden vorgestellt.

5.1.1. Accuracy und Loss

Die Accuracy ist eine der einfachsten Metriken zur Evaluierung eines Machine-Learning-Modells. Sie gibt an, wie viele der Instanzen korrekt klassifiziert wurden. Die Accuracy wird vereinfacht wie folgt berechnet:

$$\text{Accuracy} = \frac{n_{\text{richtig klassifiziert}}}{n_{\text{gesamt}}} \quad (5.1)$$

Bezug auf die in Abschnitt 5.1 beschriebenen Kategorien, kann die Accuracy wie folgt berechnet werden [17]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.2)$$

Die Accuracy wird typischerweise in Prozent angegeben und liegt zwischen 0 und 100%. Sie ermöglicht eine schnelle Einschätzung der Modellgüte. Allerdings kann die Accuracy irreführend sein, da sie die Anzahl der falsch klassifizierten Instanzen nicht berücksichtigt. Ein Modell, das alle Instanzen als negativ klassifiziert, könnte eine hohe Accuracy aufweisen, obwohl es nicht leistungsfähig ist.

Dennoch ist die Accuracy einfacher zu interpretieren als der hier vorgestellte Loss. Der Loss ist eine Metrik, die die Güte eines Modells anhand der Wahrscheinlichkeiten der Klassifikationen bewertet. Für binäre Klassifikationen wird der Binary Crossentropy Loss verwendet, der wie folgt berechnet wird [17]:

$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (5.3)$$

Symbol	Bedeutung
n	Anzahl der Trainingsbeispiele
k	Anzahl der Klassen (bei Mehrklassen-Klassifikation)
y_i	Wahre Klasse (0 oder 1) für das i -te Beispiel
$\hat{y}_i$	Vorhergesagte Wahrscheinlichkeit für Klasse 1

In jeder Epoche des Trainings werden beide Werte berechnet und stellen so die Verbesserung des Modells dar. Der Loss wird dabei minimiert, während die Accuracy maximiert wird. Bereits in der Letzten Studienarbeit wurden die getestete Modelle anhand dieser Metriken evaluiert.

5.1.2. Confusion Matrix und F1 Score

Eine aussagekräftigere Methode zur Evaluierung ist der F1 Score. Um diesen nachzuvollziehen ist zunächst eine Aufschlüsselung der klassifizierten Daten notwendig. Die Confusion Matrix ist eine Tabelle, die die Anzahl der korrekten und falschen Klassifikationen für jede Klasse anzeigt. Die Confusion Matrix hat die folgende Form [18]:

	Vorhergesagt: Positiv	Vorhergesagt: Negativ
Tatsächlich Positiv	TP	FN
Tatsächlich Negativ	FP	TN

Hier sind die Werte TP, FP, TN und FN die in Abschnitt 5.1 bereits eingeführt wurden wieder zu finden. Auf der Hauptdiagonale dieser 2×2 Matrix befinden sich die korrekt klassifizierten Instanzen, während auf der Nebendiagonale die falsch klassifizierten Instanzen zu finden sind.

Für jeden Evaluierungsauftrag lässt sich diese Matrix bestimmen. Vorteilhaft an dieser Darstellung ist, dass die Möglichkeit besteht dieses Modell auf nicht binäre Klassifikation zu erweitern.

Aus dieser Matrix lassen sich nun weitere Metriken ableiten. Eine davon ist der F1 Score. Der F1 Score ist das harmonische Mittel zwischen Precision und Recall. Precision gibt an, wie viele der als positiv klassifizierten Instanzen tatsächlich positiv sind, während Recall angibt, wie viele der tatsächlich positiven Instanzen korrekt klassifiziert wurden. Der F1 Score wird wie folgt berechnet [18]:

$$F1 = \frac{2TP}{2TP + FP + FN} \quad (5.4)$$

Der F1 Score liegt zwischen 0 und 1, wobei 1 die beste Leistung darstellt. Ein hoher

F1 Score deutet darauf hin, dass das Modell sowohl eine hohe Precision als auch einen hohen Recall aufweist. Dies sagt aus, wie gut das Modell in der Lage ist, positive Instanzen korrekt zu klassifizieren. Beispielhafte Confusion Matritzen und F1 Scores werden in Abschnitt 5.2 vorgestellt.

5.2. Reevaluierung des Modells

In diesem Kapitel soll das Ergebnismodell der letzten Studienarbeit, MobileNet V1 erneut Evaluert werden. Diesmal mithilfe der neuen Metriken (Unterabschnitt 5.1.2) und anhand eines neuen Datensatzes von PCBs (siehe Unterabschnitt 4.4.2).

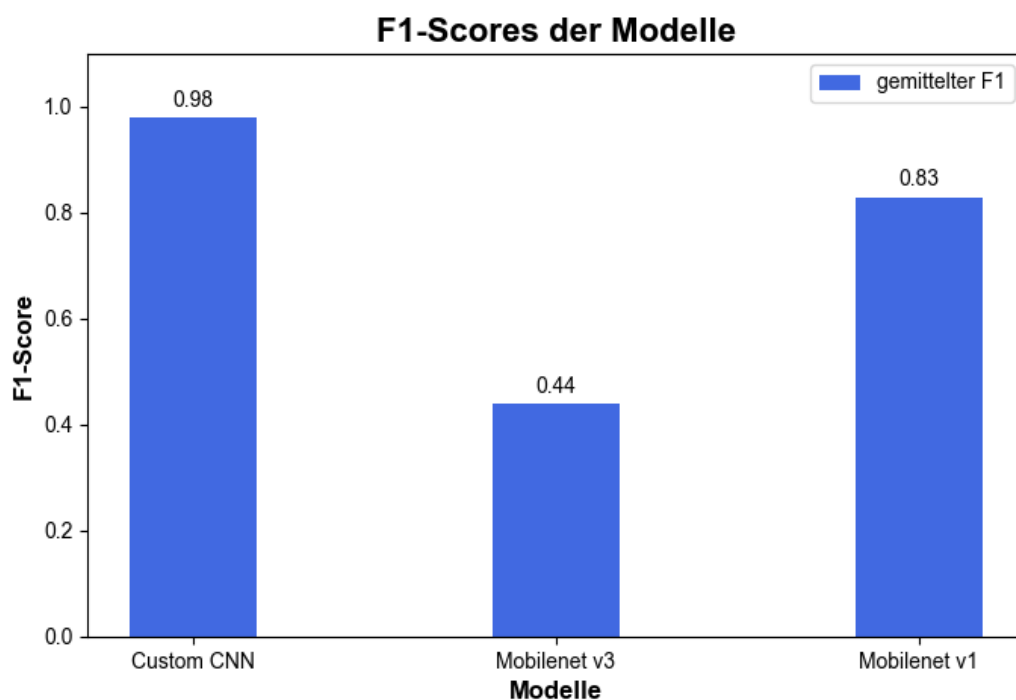


Abbildung 5.1.: Vergleich der Modelle anhand des gemittelten F1 Scores über beide Klassen

Für diese Reevaluierung wurden mehrere Modelle getestet. In dieser Arbeit werden drei Vertreter dieser Testreihe vorgestellt. Die Ergebnisse der Reevaluierung sind in Abbildung 5.1 dargestellt. Da es sich bei Mobilenet V1 und Mobilenet V3 um vortrainierte Modelle von Tensorflow handelt, soll auch ein selbstgeschriebenes Modell evaluiert

werden. Dieses Modell "Custom CNN" wurde im Rahmen einer anderen Veranstaltung entwickelt und fließt mit in diese Tests ein.

Die besten Ergebnisse liefert das Custom CNN, während Mobilenet Version 3 auch nach mehreren Optimierungsversuchen schlecht abschneidet. Dies ist Abbildung 5.1 zu entnehmen. Hier sind die gemittelten F1 Scores über beide Klassen für die verschiedenen Modelle dargestellt ist. Berechnungsgrundlage ist die in Unterabschnitt 5.1.2 vorgestellte Confusion Matrix.

Ein detaillierter Vergleich der Modelle zeigt, dass das Custom CNN auch in allen anderen Metriken überlegen ist. Aus Gründen der Übersichtlichkeit entfallen die genauen Tabellen mit allen Daten. Die vollständigen Repräsentationen sind dem Anhang Abschnitt B zu entnehmen. MobileNet V1 liefert ebenfalls gute Ergebnisse und ist aufgrund der Einfachheit der Integration und der guten Ergebnisse nicht zu vernachlässigen.

Für die folgenden Betrachtungen wurde MobileNet V3 ausgeschlossen. Seine schlechten Ergebnisse lassen auf eine schlechte Anpassung an den Datensatz oder falsche Einstellung schließen. Im Direkten Vergleich haben sowohl das Custom CNN als auch MobileNet V1 tendenziell mehr Probleme mit False Positive Klassifikationen. Dies ist in Abbildung 5.2 zu sehen.

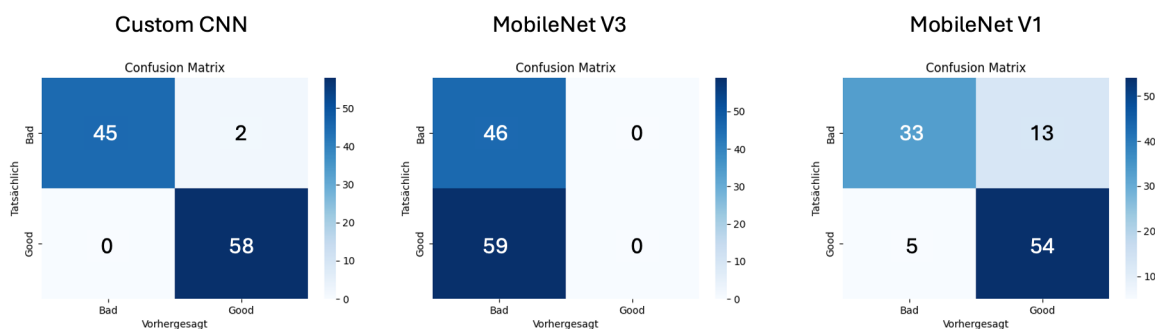


Abbildung 5.2.: Vergleich der Modelle anhand der Confusion Matrizen

Diese gemeinsamen Probleme lassen sich in diesem Fall auf den Datensatz zurückführen. Die falsch Positiv Klassifizierten Bilder sind ebenfalls für den Menschen nur schwer zu klassifizieren, da ein Stück Leiterbahn fehlt oder eine schlecht belichtete Komponente fehlt (PCBs siehe Abbildung 4.3 Rechts). In den Praxistests während des

Probetriebs im Labor hat sich gezeigt, dass MobileNet V1 in allen beobachteten Fällen in der Lage ist eine korrekte Klassifikation durchzuführen.

Zusammenfassend lässt sich sagen, dass das Custom CNN in allen getesteten Metriken überlegen ist. Aufgrund der bestehenden zeitlichen Einschränkungen und der bestehenden Programmpipeline wird jedoch weiterhin MobileNet V1 verwendet. Eine neu trainierte Version mit dem aktuellen Datensatz ersetzt fortan das Modell der Machbarkeitsstudie.

6. Fazit und Ausblick

In dieser Arbeit wurde ein bestehendes Projekt zur Bildklassifikation von Leiterplattenfehlern weiterentwickelt und evaluiert. Der Schwerpunkt lag dabei auf der Implementierung einer Schnittstelle zur besseren Verwendbarkeit und Präsentierbarkeit des bestehenden Projekts. Darüber hinaus wurden verschiedene Modelle und Methoden zur Bildklassifikation anhand eines neuen Datensatzes untersucht und miteinander verglichen.

Trotz der erzielten Fortschritte gibt es mehrere Bereiche, die für zukünftige Arbeiten von Interesse sein könnten. Eine mögliche Erweiterung besteht darin, die Integration des Custom CNN in die Programmpipeline zu ermöglichen. Darüber hinaus könnten weitere Modelle und Methoden zur Bildklassifikation untersucht und verglichen werden, um die bestmögliche Leistung zu erzielen.

Ergebnis ist eine funktionierende Webanwendung, die es ermöglicht, neue Bilder des FESTO CP-Lab darzustellen, zu klassifizieren und die Ergebnisse auf einer Weboberfläche zu visualisieren. Die Anwendung koordiniert die Überwachung eines Ordners für neue Bilder und die Klassifizierung dieser Bilder in defekt oder nicht defekt. Die Implementierung erfolgte in Python unter Verwendung der Flask-Bibliothek für die Webanwendung und der TensorFlow-Bibliothek für die Bildklassifikation. Der Quellcode für dieses Projekt ist auf GitHub verfügbar und kann von Interessierten eingesehen und weiterentwickelt werden. Der Link zu diesem Repository und einer Anleitung ist im Anhang Abschnitt A zu finden.

Eine weitere mögliche Fortsetzung dieser Studienarbeit wäre die Implementierung von Ensemble Methoden, bei denen mehrere Modelle kombiniert werden, um die Klassifikationsergebnisse zu verbessern. Dies könnte insbesondere bei komplexen Datensätzen von Vorteil sein, bei denen einzelne Modelle möglicherweise nicht die bestmögliche Leistung erzielen.

Zukünftige Arbeiten sollten sich darauf konzentrieren, die Integration des Custom CNN in die Programmpipeline zu ermöglichen, um die Leistung der Klassifikation weiter zu verbessern.

Literaturverzeichnis

- [1] W. G. Kropatsch, N. Artner und M. Gobel, „Einführung in Visual Computing II: Grundlagen der Digitalen Bildverarbeitung Skriptum zur Vorlesung,“
- [2] Finbridge.de. „Computer Vision für Finance [Teil 2]: Convolutional Neural Networks,“ Finbridge GmbH & Co KG. (8. Sep. 2022), Adresse: <https://www.finbridge.de/ml-artikel/2022/09/08/computer-vision-fuer-finance-teil-2> (besucht am 01.03.2025).
- [3] Intel. „Convolutional Neural Networks (CNN) und Deep Learning,“ Intel. (), Adresse: <https://www.intel.com/content/www/de/de/internet-of-things/computer-vision/convolutional-neural-networks.html> (besucht am 01.03.2025).
- [4] „Machine Learning Regression,“ Mailchimp. (), Adresse: <https://mailchimp.com/de/resources/machine-learning-regression/> (besucht am 10.11.2024).
- [5] H. Süße und E. Rodner, *Bildverarbeitung und Objekterkennung: Computer Vision in Industrie und Medizin*. Wiesbaden: Springer Fachmedien Wiesbaden, 2014, ISBN: 978-3-8348-2605-3 978-3-8348-2606-0. DOI: 10.1007/978-3-8348-2606-0. Adresse: <https://link.springer.com/10.1007/978-3-8348-2606-0> (besucht am 15.10.2024).
- [6] S. Mittelstand. „Erste Schritte mit Python HTTP-Anfragen für REST-APIs.“ (), Adresse: <https://www.datacamp.com/tutorial/making-http-requests-in-python> (besucht am 01.03.2025).
- [7] K. Pykes. „Programmieren mit Python APIs: Ihr ultimativer Guide in einfachen Schritten.“ (), Adresse: <https://www.software-mittelstand.info/apis-mit-python-programmieren-eine-schritt-fuer-schritt-anleitung/> (besucht am 01.03.2025).
- [8] D. O. LLC. „Erste schritte mit der requests-bibliothek in python | DigitalOcean.“ (), Adresse: <https://www.digitalocean.com/community/tutorials/how-to-get-started-with-the-requests-library-in-python-de> (besucht am 01.03.2025).

- [9] C. Rodríguez, M. Baez, F. Daniel u. a., „REST APIs: A large-scale analysis of compliance with principles and best practices,“ in *Web Engineering*, A. Bozzon, P. Cudre-Maroux und C. Pautasso, Hrsg., Cham: Springer International Publishing, 2016, S. 21–39, ISBN: 978-3-319-38791-8. DOI: 10.1007/978-3-319-38791-8_2.
- [10] Flask. „Welcome to Flask — Flask Documentation (3.1.x).“ (), Adresse: <https://flask.palletsprojects.com/en/stable/> (besucht am 01.03.2025).
- [11] *Diskussion über die Projektstrukturen von Python ML Projekten*, unter Mitarb. von M. B. Gür, 8. Nov. 2024.
- [12] P. Oliveira. „How to write a python configuration file,“ LambdaTest. Section: Selenium Python. (29. Sep. 2023), Adresse: <https://www.lambdatest.com/blog/python-configuration-file/> (besucht am 02.03.2025).
- [13] *Model View Controller*, in *Wikipedia*, Page Version ID: 248816402, 22. Sep. 2024. Adresse: https://de.wikipedia.org/w/index.php?title=Model_View_Controller&oldid=248816402 (besucht am 02.03.2025).
- [14] „Threading — thread-based parallelism,“ Python documentation. (), Adresse: <https://docs.python.org/3/library/threading.html> (besucht am 11.03.2025).
- [15] P. S. Foundation. „Venv — creation of virtual environments,“ Python documentation. (), Adresse: <https://docs.python.org/3/library/venv.html> (besucht am 12.03.2025).
- [16] S. I. GmbH, „VISOR Benutzerhandbuch,“ Jan. 2019.
- [17] A. Wiki. „Accuracy and loss | AI wiki.“ (17. Dez. 2019), Adresse: <https://machine-learning.paperspace.com/wiki/accuracy-and-loss> (besucht am 02.03.2025).
- [18] Z. C. Lipton, C. Elkan und B. Narayanaswamy, *Thresholding Classifiers to Maximize F1 Score*, 14. Mai 2014. DOI: 10.48550/arXiv.1402.1892. arXiv: 1402.1892[stat]. Adresse: <http://arxiv.org/abs/1402.1892> (besucht am 02.03.2025).

A. Anhang

A. Anleitung zur Verwendung

Nach der korrekten Installation des Programms, welches in Kapitel 4.4 beschrieben ist, kann das Programm wie folgt verwendet werden: Die Installation ist zum Zeitpunkt der Übergabe dieser Arbeit bereits auf dem FESTO-PC erfolgt.

Der Quellcode ist jederzeit unter folgender URL erreichbar: https://github.com/andreasbraig/DHBWMA_HHAT_Image-Classification

Im Branch main befindet sich die Version zum Zeitpunkt der Abgabe.

Zunächst sollte auf folgende Punkte geachtet werden:

- Die Installation ist auf dem FESTO-PC erfolgt. Das Passwort ist bei Frau Prof. Dr.-Ing. Bozena Lamek-Creutz zu erfragen.
- Die verwendete Python Version ist 3.12.9.
- Alle verwendeten Bibliotheken sind in der requirements.txt Datei aufgelistet.
- Das Programmverzeichnis mit allen Daten liegt unter:

`D:/Repo/DHBWMA_HHAT_Image-Classification`

Um das Programm auszuführen, bitte der folgenden Anleitung folgen:

1. Das PowerShell Skript auf dem Desktop mit dem Namen `Image_classification.ps1` ausführen.
2. Sobald in PowerShell die URL `http://127.0.0.1:5000` angezeigt wird, kann die Webseite im Browser geöffnet werden.

3. Wenn mit der VISOR Kamera der FESTO CP-Lab Kamera Fotos archiviert werden, sollten diese auf der Website mit Klassifizierungsergebnis dargestellt werden.

Im Folgenden wird in zwei Methoden erklärt, wie Fotos mit der VISOR Kamera archiviert werden können:

A.1. Manuelle Archivierung

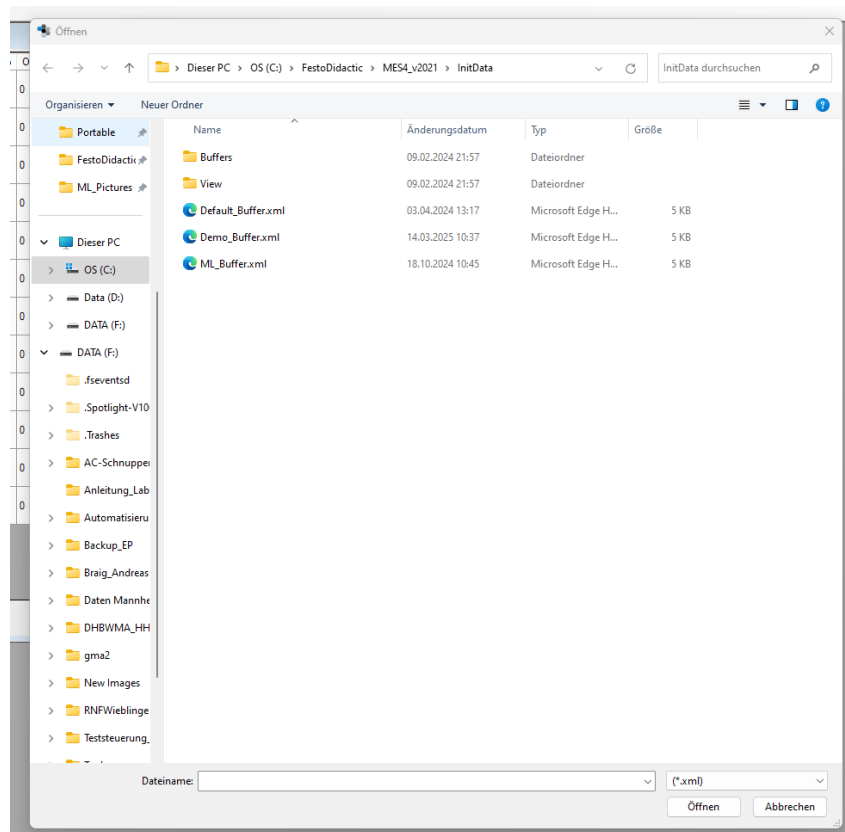
1. Die FESTO CP-Lab Anlage muss mit Strom und Druckluft versorgt sein.
2. Auf dem FESTO-PC die Visor Software starten.
3. Zunächst Config öffnen, das Passwort ist `Admin` für den Benutzer `admin`.
4. Dieses Fenster kann direkt wieder geschlossen werden. (die Einstellungen sind bereits gespeichert, also `Yes` drücken)
5. Im Visor Fenster auf `View` klicken und unten links `Archiving` durch Klicken aktivieren.
6. Nun sollte die Kamera Fotos aufnehmen und diese in folgenden Ordner ablegen:

`D:/Repo/DHBWMA_HHAT_Image-Classification/Bilder/new`

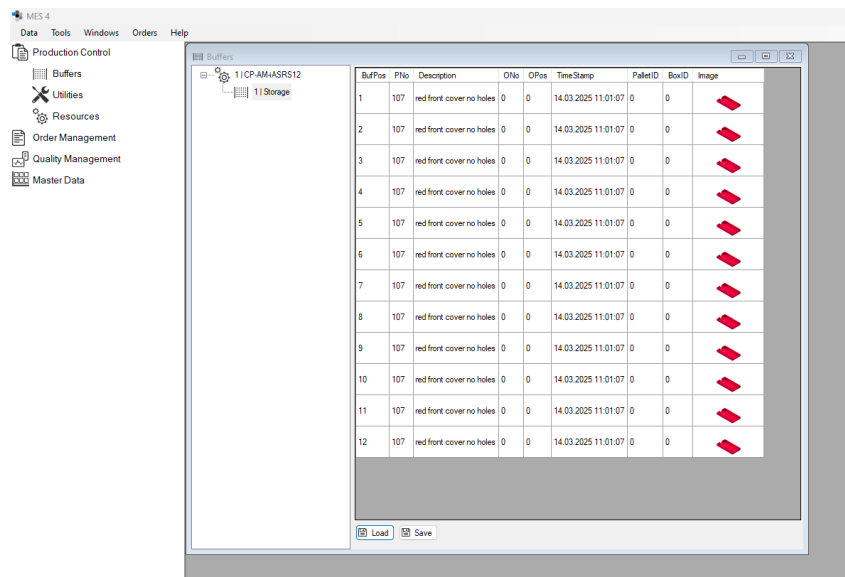
A.2. Automatische Archivierung

Für diese Methode wurde in der `MES 4` Software der FESTO CP-Lab Anlage eine neue Order mit einem neuen Buffer erstellt, welcher den Job für die Kamera enthält.

1. Die FESTO CP-Lab Anlage muss mit Strom und Druckluft versorgt sein.
2. Die Software `MES 4` auf dem FESTO-PC starten.

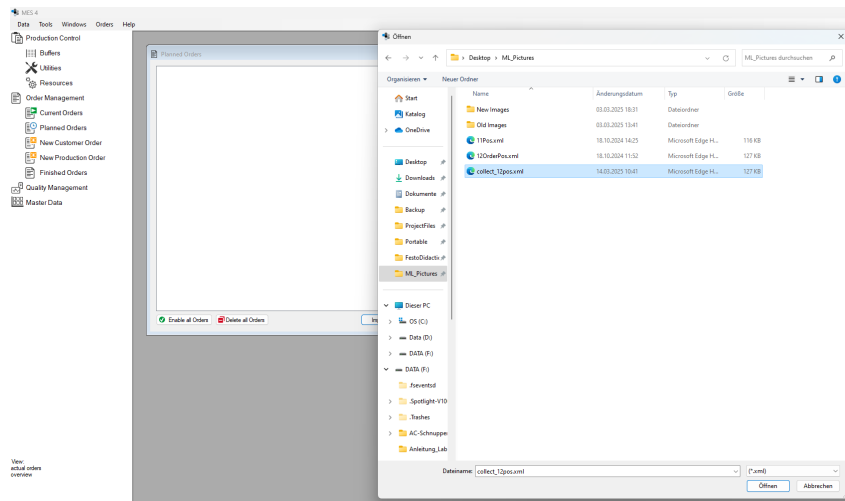


3. In der MES 4 Software einen neuen Buffer laden, dieser ist mit dem Namen Demo_Buffer versehen.



4. Der Buffer sollte jetzt mit pinken Boxen gefüllt sein.

5. Über das **Planned Orders Menü** eine neue Order importieren. Diese liegt unter **C:/Users/Festo/Desktop/ML Pictures** ab.



6. Die Order sollte jetzt in der MES 4 Software sichtbar sein und muss gestartet werden.

7. Nun sollte die Kamera Fotos aufnehmen und diese in folgenden Ordner ablegen:

D:/Repo/DHBWMA_HHAT_Image-Classification/Bilder/new

8. Zur Kontrolle kann auf dem HMI der Kamerastation der aktuell ausgewählte Job betrachtet werden. Dieser sollte 7 sein.

B. Performance der Modelle

B.1. Pytorch Custom CNN

Label	F1-Score			Support
	Precision	Recall	F1-Score	
Bad	1.00	0.96	0.98	47
Good	0.97	1.00	0.98	58
Accuracy			0.98	105
Macro avg	0.98	0.98	0.98	105
Weighted avg	0.98	0.98	0.98	105

Tabelle A.1.: Performance der Custom CNN (PyTorch) Modell

B.2. MobileNet V3 (TensorFlow)

Label	F1-Score			Support
	Precision	Recall	F1-Score	
Bad	0.44	1.00	0.61	46
Good	0.00	0.00	0.00	59
Accuracy			0.44	105
Macro avg	0.22	0.50	0.30	105
Weighted avg	0.19	0.44	0.27	105

Tabelle A.2.: Performance der MobileNet V3 (TensorFlow) Modell

B.3. MobileNet V1 (TensorFlow)

Label	F1-Score			Support
	Precision	Recall	F1-Score	
Bad	0.87	0.72	0.79	46
Good	0.81	0.92	0.86	59
Accuracy			0.83	105
Macro avg	0.84	0.82	0.82	105
Weighted avg	0.83	0.83	0.83	105

Tabelle A.3.: Performance der MobileNet V1 (TensorFlow) Modell